

Compiler Design (CS24301) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PYQ BANK

Compiler Design (CS24301)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

Complete Course Structure & Syllabus Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Lexical Analysis & Automata	6 Phases, Two-Buffer Scheme, Thompson RE to NFA, Subset Construction, Hopcroft DFA Minimization
Module II	Syntax Analysis & Parsers	CFG, Left Recursion/Factoring, FIRST & FOLLOW, LL(1) Table, Shift-Reduce, LR(0), SLR(1), CLR(1), LALR(1)
Module III	Semantic Analysis & Types	Syntax-Directed Definitions (SDD), Synthesized/Inherited, S/L-Attributed, Type Checking, Symbol Tables
Module IV	Intermediate Code & Runtime	Three-Address Code (Quadruples/Triples), Multi-D Array Addressing, Backpatching, Activation Records
Module V	Optimization & Code Gen	Basic Blocks, CFG, DAG, Loop Invariant Code Motion, Reaching Definitions, Register Coloring

Exam Preparation & High-Yield Guide

This publication-grade master book consolidates all 5 modules with formal mathematical proofs, KaTeX-rendered formulas, step-by-step worked traces on classic datasets, and comprehensive model answers to BIT Mesra end-semester examination questions.

☰ Module I: Lexical Analysis — Complete 7-Topic Syllabus Tracker

Topic 1: Introduction to Compilers & Its Cousins (P-I-C-A-L)

Topic 2: Structure of a Compiler (Phases, Symbol Table, Errors)

Topic 3: Lexical Analyzer (Token, Lexeme, Pattern Triad)

Topic 4: Input Buffering (Two-Buffer Scheme, Sentinels)

Topic 5: Specification of Tokens (Regular Expressions & Operators)

Topic 6: Recognition of Tokens (Transition Diagrams & DFA)

Topic 7: DFA Directly from Regular Expressions (FLF Rules)

Exam Prep: Active Recall Check & Exam-Important Questions

Topic 1: Introduction to Compilers and its Cousins

1. What is a Compiler?

A **compiler** is a specialized software program that translates a source program written in a high-level language into an equivalent target program, usually in machine code or assembly language.

High-Level Language (C, C++, Java) → [COMPILER] → Target Machine / Assembly Code

Parameter	Compiler	Interpreter
Translation Mode	Translates the entire source program into target machine code before execution.	Translates and executes the source program statement-by-statement at runtime.
Execution Speed	Very fast target execution once compiled (standalone binary).	Slower execution due to continuous interpretation overhead.
Memory Requirement	Generates intermediate object files; requires memory for full compilation.	Minimal initial memory; does not produce a standalone object binary.
Error Reporting	Reports all syntactic & semantic errors after full program scan.	Halts execution at the very first line containing an error.

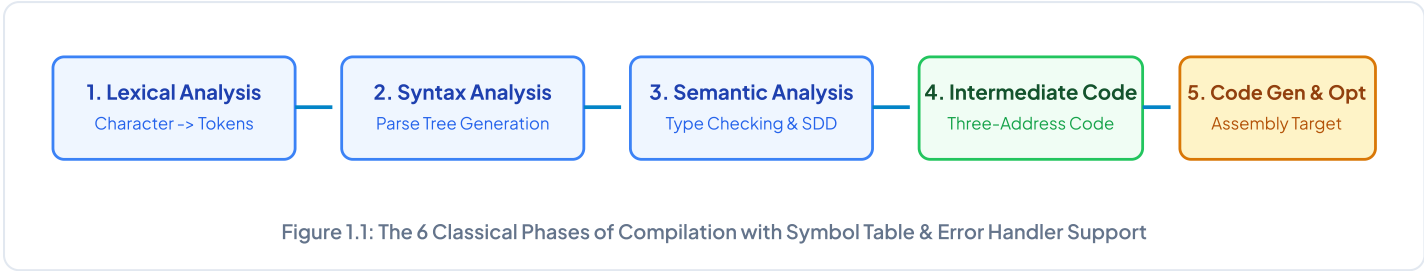
2. Cousins of the Compiler (Language Processing Ecosystem):

- 1. Preprocessor:** Handles source-level directives before compilation (e.g., `#include`, `#define` macro expansion, file inclusion, conditional compilation `#ifdef`).
- 2. Assembler:** Translates human-readable assembly mnemonics (e.g., `MOV R1, R2`) into relocatable binary machine code.
- 3. Linker:** Resolves external cross-file symbol references and links external precompiled library binaries into a single executable module.
- 4. Loader:** Dynamically loads the executable binary into physical RAM, allocates stack/heap segments, adjusts relocatable memory addresses, and initializes the CPU Program Counter (PC).

🧠 Memory Hook: P-I-C-A-L

Preprocessor → Compiler → Assembler → Linker → Loader

Topic 2: Structure of a Compiler (Front End vs. Back End)



Cross-Phase Supporting Components:

Symbol Table Management: A centralized data structure (e.g., chained hash table) storing variable and function identifiers, data types, memory offsets, scope levels, and register assignments.

Error Detection & Handling: Detects lexical errors (invalid characters), syntax errors (mismatched parentheses), and semantic errors (type mismatches) and provides recovery mechanisms to continue parsing without crashing.

Topic 3: Lexical Analyzer (Tokens, Lexemes, Patterns)

Concept	Definition	Concrete Example
Token	An abstract syntactic category/class passed to the parser.	`IDENTIFIER`, `NUMBER`, `ASSIGN_OP`, `KEYWORD`
Lexeme	The actual concrete sequence of characters in the source code matching a pattern.	`count`, `100`, `=`, `while`
Pattern	The formal grammar/regular expression rule describing valid lexemes.	`letter(letter digit)*`, `[0-9]+`

Example: Lexical Tokenization Breakdown for `count = 25;`

Lexeme	Token Name	Attribute Value (Symbol Table Pointer / Numeric Constant)
`count`	`IDENTIFIER`	Pointer to Symbol Table entry for `count`
`=`	`ASSIGN_OP`	None
`25`	`NUMBER`	Integer value `25`
`;`	`SEMICOLON`	None

Topic 4: Input Buffering (Two-Buffer Scheme & Sentinels)

Reading one character at a time directly from secondary storage causes severe I/O bottlenecks. Lexical analyzers use an **Input Buffering** scheme with block reads:

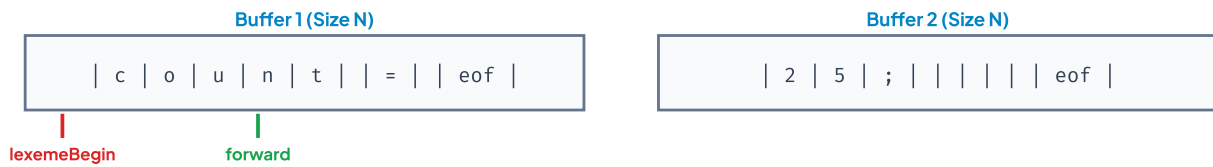


Figure 1.2: Two-Buffer Scheme with lexemeBegin and forward pointers & eof Sentinels

`lexemeBegin` Pointer: Marks the start of the current lexeme being recognized.

`forward` Pointer: Scans ahead one character at a time until a token boundary is identified.

Sentinels (`eof`): Appending an `eof` character at the end of each buffer reduces two boundary tests per character (end of buffer check + EOF check) down to a single test.

Topic 5 & 6: Specification & Recognition of Tokens

Regular Expression Operators & Precedence

1. **Kleene Star (*)**: Zero or more repetitions (Highest Precedence).
2. **Concatenation**: Sequence of symbols (Medium Precedence).
3. **Alternation (|)**: Logical OR / choice (Lowest Precedence).

Topic 7: DFA Directly from Regular Expressions (Direct Method)

The Direct Method constructs a minimal DFA directly from an augmented regular expression $(r)\#$ without generating an intermediate Thompson NFA.

The 4 Fundamental Functions: `nullable`, `firstpos`, `lastpos`, `followpos`

Node Type n	`nullable(n)`	`firstpos(n)`	`lastpos(n)`
Leaf ϵ	true	$\emptyset$	$\emptyset$
Leaf Position i	false	$\{i\}$	$\{i\}$
OR: $c_1 \mid c_2$	`nullable(c1)    nullable(c2)`	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
Cat: $c_1 \cdot c_2$	`nullable(c1) && nullable(c2)`	if nullable(c_1) then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if nullable(c_2) then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
Star: c_1^*	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

`followpos(i)` Invariant Rules:

1. For Cat node $n = c_1 \cdot c_2$: For every position $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_2)$.
2. For Star node $n = c_1^*$: For every position $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_1)$.

Step-by-Step Solved Problem: Direct DFA Construction for $(a \mid b)^*abb\#$

Step 1: Assign Position Indices: $(a_1 \mid b_2)^* \cdot a_3 \cdot b_4 \cdot b_5 \cdot \#_6$

Step 2: Compute 'followpos' Table:

Node Position i	Symbol	'followpos(i)'
1	a	$\{1, 2, 3\}$
2	b	$\{1, 2, 3\}$
3	a	$\{4\}$
4	b	$\{5\}$
5	b	$\{6\}$
6	$\#$	$\emptyset$

Step 3: DFA States & Transitions:

- $S_0 = \text{firstpos}(\text{root}) = \{1, 2, 3\}$
- $\delta(S_0, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_1$
- $\delta(S_0, b) = \text{followpos}(2) = \{1, 2, 3\} = S_0$
- $\delta(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_1$
- $\delta(S_1, b) = \text{followpos}(2) \cup \text{followpos}(4) = \{1, 2, 3, 5\} = S_2$
- $\delta(S_2, b) = \text{followpos}(2) \cup \text{followpos}(5) = \{1, 2, 3, 6\} = S_3$ (Accepting, since $6 \in S_3$)

M1 Active Recall & Exam-Important Question Bank

Q1. Explain the 6 phases of a compiler with a neat translation diagram for the statement 'position = initial + rate * 60'. (10 Marks)

- Lexical Analysis:** Produces token stream: $\langle \text{id}_1 \rangle \langle = \rangle \langle \text{id}_2 \rangle \langle + \rangle \langle \text{id}_3 \rangle \langle * \rangle \langle 60 \rangle$.
- Syntax Analysis:** Builds parse tree showing operator precedence ($*$ over $+$).
- Semantic Analysis:** Checks types and inserts 'inttofloat(60)' conversion.
- Intermediate Code Generation:** Emits Three-Address Code:
 $\text{'t1 = inttofloat(60); t2 = id3 * t1; t3 = id2 + t2; id1 = t3'}$
- Code Optimization:** Constant propagation & strength reduction: $\text{'t1 = id3 * 60.0; id1 = id2 + t1'}$
- Code Generation:** Emits target assembly: $\text{'LDF R2, id3; MULF R2, \#60.0; ADDF R2, id2; STF id1, R2'}$.

Q2. Explain the Two-Buffer scheme and the role of Sentinels in input buffering. (8 Marks)

- Two-Buffer Scheme:** Uses two contiguous blocks of size N in memory. While the scanner processes characters in Buffer 1 using 'lexemeBegin' and 'forward' pointers, Buffer 2 can be asynchronously filled via a single system I/O read call.
- Role of Sentinels:** Without sentinels, testing every character requires two comparisons: (1) 'if (forward == buffer_end)' and (2) test character. Placing an 'eof' sentinel at the end of each buffer reduces this to a single check: only when an 'eof' character is encountered does the scanner check whether it has reached a buffer boundary or the genuine end of the source file.

Module II: Syntax Analysis — Complete 10-Topic Syllabus Tracker

Topic 1: Introduction to Syntax Analysis & CFG Grammars

Topic 3: Recursive Top-Down Parsers (Recursive Descent & Backtracking)

Topic 5: Design of LL(1) Parser (FIRST & FOLLOW Inductive Rules)

Topic 7: Variants of LR Parsers (LR(0), SLR(1), CLR(1), LALR(1))

Topic 9: Detection of Syntax Errors (Panic-Mode & Phrase-Level)

Topic 2: Grammar Rewriting (Left Recursion & Left Factoring)

Topic 4: Non-Recursive Top-Down Parsers (Predictive Stack & Buffer)

Topic 6: Bottom-Up Parsers (Shift, Reduce & Handle Pruning)

Topic 8: Handling Parsing Conflicts (Shift-Reduce & Reduce-Reduce)

Topic 10: Reporting of Syntax Errors & Informative Diagnostics

Topic 1: Introduction to Syntax Analysis & CFG Grammars

Syntax Analysis (Parsing) takes the stream of tokens generated by the lexical analyzer and verifies whether the token sequence conforms to the syntactic rules of the language defined by a **Context-Free Grammar (CFG)** $G = (V, T, P, S)$.

Topic 2: Grammar Rewriting Transformations

1. Immediate Left Recursion Elimination:

For grammar production $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2$ where no β_i starts with A :

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon \end{aligned}$$

2. Left Factoring:

Delays the parsing decision when multiple productions share a common prefix α :

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \implies A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

Topic 3 & 4: Top-Down Parsers (Recursive vs. Non-Recursive)

Top-Down Parser Type	Internal Architecture & Mechanism	Key Limitations & Mitigations
Recursive Descent Parser	Executes a set of mutually recursive C/Java functions where each non-terminal corresponds to a procedure.	Cannot directly handle left recursion (causes infinite loop); requires backtracking without predictive lookahead.
Non-Recursive Predictive Parser	Uses an explicit Parser Stack, an Input Buffer with endmarker '\$', and a $2D$ parsing table $M[A, a]$.	Requires grammar strictly to be $LL(1)$ without conflicts.

Topic 5: Design of LL(1) Parsers (FIRST & FOLLOW Inductive Formulations)

Inductive Rules for FIRST(α): 1. If X is a terminal, $\text{FIRST}(X) = \{X\}$. 2. If $X \rightarrow \epsilon$, then $\epsilon \in \text{FIRST}(X)$. 3. If $X \rightarrow Y_1 Y_2 \dots Y_k$, add all non- ϵ symbols of $\text{FIRST}(Y_1)$ to $\text{FIRST}(X)$. If $\epsilon \in \text{FIRST}(Y_1)$, add $\text{FIRST}(Y_2)$, and so on. If all $Y_i \Rightarrow^* \epsilon$, add $\epsilon \in \text{FIRST}(X)$.

Inductive Rules for FOLLOW(A): 1. For start symbol S , place endmarker $\$$ into $\text{FOLLOW}(S)$. 2. For production $A \rightarrow \alpha B \beta$, add $(\text{FIRST}(\beta) \setminus \{\epsilon\})$ into $\text{FOLLOW}(B)$. 3. For production $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where $\epsilon \in \text{FIRST}(\beta)$, add all elements of $\text{FOLLOW}(A)$ into $\text{FOLLOW}(B)$.

Condition for Grammar to be LL(1)

A grammar G is $LL(1)$ if and only if for every pair of productions $A \rightarrow \alpha \mid \beta$: 1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ 2. If $\epsilon \in \text{FIRST}(\beta)$, then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$.

Topic 6 & 7: Bottom-Up Parsers & LR Variants Hierarchy

Bottom-Up Parsers construct the parse tree starting from the terminal leaves at the bottom and reducing back to the start symbol S at the root (Rightmost Derivation in Reverse):

LR Family	Item Representation	Reduction Decision Rule	State Complexity & Relative Power
1. LR(0)	$[A \rightarrow \alpha \cdot \beta]$	Reduce unconditionally in entire row for all input tokens.	Smallest table; least powerful. Prone to conflicts.
2. SLR(1)	$[A \rightarrow \alpha \cdot \beta]$ (Same as LR(0))	Reduce only on input tokens $a \in \text{FOLLOW}(A)$.	Same number of states as LR(0); eliminates many conflicts.
3. CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$ (With 1 Lookahead)	Reduce only on exact propagated lookahead token a .	Most powerful; massive number of states (thousands).
4. LALR(1)	Merged CLR(1) items with identical cores.	Reduce on merged lookahead sets.	Same state count as SLR(1) with near CLR(1) parsing power. Used in Yacc / Bison.

Topic 8 – 10: Handling Parsing Conflicts & Error Recovery

1. Parsing Conflicts:

Shift-Reduce (S-R) Conflict: The parser cannot decide whether to shift the incoming token or reduce the stack handle (e.g., the classic Dangling Else ambiguity).

Reduce-Reduce (R-R) Conflict: The parser has two distinct valid reduction rules available for the same lookahead token, indicating deep grammar ambiguity.

2. Syntax Error Recovery Techniques:

Panic-Mode Recovery: Parser discards input tokens one by one until a designated *synchronizing token* (e.g., `;` or `}`) is found, then pops non-terminals to resume parsing.

Phrase-Level Recovery: Parser performs local correction on the remaining input (e.g., inserting a missing semicolon or replacing a comma with a semicolon).

Error Productions: Grammar is augmented with explicit rules that anticipate common syntactic mistakes (e.g., `5x` instead of `5 \* x`).



Q1. Given grammar $S \rightarrow (L) \mid a$ and $L \rightarrow L, S \mid S$, eliminate left recursion and compute FIRST and FOLLOW. (8 Marks)

1. Eliminate Left Recursion on L :

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \epsilon$$

2. Compute FIRST Sets:

$$\text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L) = \text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L') = \{ ,, \epsilon \}$$

3. Compute FOLLOW Sets:

$$\text{FOLLOW}(S) = \{ \$,), , \}$$

$$\text{FOLLOW}(L) = \{ \}$$

$$\text{FOLLOW}(L') = \text{FOLLOW}(L) = \{ \}$$

Module III: Semantic Analysis & Intermediate Code — Complete 9-Topic Syllabus Tracker

Topic 1: Introduction to Semantic Analysis (Static vs Dynamic Checks)

Topic 3: Syntax-Directed Translation Schemes (SDTS Actions)

Topic 5: Three Address Code (Quadruples, Triples, Indirect Triples)

Topic 7: Type Checking for Expressions (Equivalence & Coercion)

Topic 9: Translation of Multi-Dimensional Array References

Topic 2: Syntax-Directed Definitions (SDD Semantic Rules)

Topic 4: SDTS for Declaration Processing & Symbol Tables

Topic 6: Types of Attributes (Synthesized vs. Inherited)

Topic 8: Intermediate Code Generation for Assignment Statements

Topic 1: Introduction to Semantic Analysis

Semantic Analysis verifies that the syntax tree conforms to the semantic rules of the language. While syntax ensures correct grammatical structure, semantics verifies logical validity:

Static Semantic Checks (Compile-Time): Type consistency, variable declared before use, scope resolution, identifier uniqueness.

Dynamic Semantic Checks (Run-Time): Division by zero, array out-of-bounds, null pointer dereferencing.

Topic 2 – 4: SDD, SDTS & Declaration Processing

Feature	Syntax-Directed Definition (SDD)	Syntax-Directed Translation Scheme (SDTS)
Representation	Context-Free Grammar + Semantic Rules associated with productions.	Context-Free Grammar with semantic action code embedded inside productions `{...}`.
Execution Timing	Specifies <i>what</i> value to compute (order determined by Dependency Graph topological sort).	Specifies <i>when</i> and in what exact sequence actions execute during parsing.

Step-by-Step SDTS: Processing Declarations `int a, b, c;`

```
D → T { L.in = T.type } L
T → int { T.type = integer }
T → float { T.type = real }
L → { L1.in = L.in } L1, id { enter(id.name, L.in) }
L → id { enter(id.name, L.in) }
```

Uses inherited attribute *L.in* to propagate base type down to all declared identifier names.

Topic 5: Three Address Code (TAC) Representations

TAC Form	Internal Data Fields	Key Advantages & Tradeoffs
1. Quadruples	<code>`(op, arg1, arg2, result)`</code> with explicit temporary names (<code>`t1`, `t2`</code>).	Easy to reorder and optimize; consumes extra memory for names.
2. Triples	<code>`(op, arg1, arg2)`</code> where intermediate results are referenced by index <code>`(0)`</code> .	Space-efficient; moving instructions requires updating all dependent indices.
3. Indirect Triples	Array of pointers to separate triple structures.	Optimizers reorder instructions by swapping array pointers without modifying triples.

Topic 6 & 7: Types of Attributes & Type Checking

1. Synthesized vs. Inherited Attributes:

Synthesized Attributes: Computed strictly from children nodes in parse tree ($A.s = f(Y_1.a, \dots, Y_k.a)$). Evaluated bottom-up in LR parsing.

Inherited Attributes: Computed from parent and/or left sibling nodes ($Y_j.i = f(A.a, Y_1.a, \dots, Y_{j-1}.a)$). Evaluated top-down.

2. Type Conversions & Equivalence:

Name Equivalence: Two types are equal if and only if they share the exact same named identifier.

Structural Equivalence: Two types are equal if they have identical internal field layouts and primitive types.

Widening (Coercion): ``int`` $\rightarrow$ ``float`` (implicit precision preservation).

Narrowing (Casting): ``float`` $\rightarrow$ ``int`` (explicit potential precision loss).

Topic 8 & 9: Intermediate Code Generation for Arrays & Assignments

1. 1D Array Reference Address Formula:

$$\text{Address}(A[i]) = \text{base} + (i - \text{low}) \times w$$

2. 2D Array Reference Address Formula (Row-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base} + ((i_1 - \text{low}_1) \times n_2 + (i_2 - \text{low}_2)) \times w$$

Where $n_2 = \text{high}_2 - \text{low}_2 + 1$ is the number of columns, and w is element byte width.

3. General N -Dimensional Row-Major Linear Offset:

$$\text{Offset} = \left(\dots ((i_1 \cdot d_2 + i_2) \cdot d_3 + i_3) \dots \right) \cdot d_k + i_k$$

$$\text{Address} = \text{base} + \text{Offset} \times w$$



Q1. Write Quadruples and Triples for statement $a = b * -c + b * -c$. (8 Marks)

Quadruples:

- (0) $\text{uminus}, c, -, t1$
- (1) $*, b, t1, t2$
- (2) $\text{uminus}, c, -, t3$
- (3) $*, b, t3, t4$
- (4) $+, t2, t4, t5$
- (5) $=, t5, -, a$

Triples:

- (0) $\text{uminus}, c, -$
- (1) $*, b, (0)$
- (2) $\text{uminus}, c, -$
- (3) $*, b, (2)$
- (4) $+, (1), (3)$
- (5) $\text{assign}, a, (4)$

Module IV: Intermediate Code & Runtime Environment — Complete 9-Topic Syllabus Tracker

- Topic 1: Complete Evaluation of Boolean Expressions
- Topic 2: Partial Evaluation (Short-Circuit) of Boolean Expressions
- Topic 3: Translation of Control Flow Constructs ('if-else', 'while')
- Topic 4: Resolution of Forward Jumps & Backpatching Algorithms
- Topic 5: Resolution of Backward Jumps in Loop Constructs
- Topic 6: Translation of Function Calls ('param', 'call', 'return_val')
- Topic 7: Translation of Function Returns & Caller Restoration
- Topic 8: Memory Layout of Code and Data (Text, Static, Heap, Stack)
- Topic 9: Activation Records (Stack Frame Anatomy: P-R-C-A-S-L-T)

Topic 1 & 2: Boolean Expressions (Complete vs. Short-Circuit Evaluation)

Evaluation Strategy	Operational Semantics	Key Advantages & Tradeoffs
1. Complete Evaluation	Evaluates all sub-expressions unconditionally before applying logical operators (e.g., bitwise '&' and ' ').	Evaluates side-effects in all operands; can cause runtime crashes on null-pointer guards (e.g., 'p != NULL && p->val == 5').
2. Partial Evaluation (Short-Circuit)	Stops evaluating as soon as the final truth value is guaranteed: - In 'A && B': If A is false, B is never evaluated. - In 'A B': If A is true, B is never evaluated.	Highly optimized execution speed; safely handles null-pointer guards and range checks.

Topic 3 – 5: Control Flow Translation & Jump Resolutions

Three-Address Code for 'if (a < b) x = 1; else x = 2;'

```
if a < b goto L1
goto L2
L1: x = 1
goto L3
L2: x = 2
L3: /* End of If-Else */
```

Backpatching Mechanics ('makelist', 'merge', 'backpatch'):

Forward Jumps: Jump targets point to instructions not yet generated. The compiler emits placeholder jumps ('goto ?') and maintains lists of pending jump instructions.

Backward Jumps: Jump targets point to previously emitted labeled instructions (common in loop repeat steps). Target address is immediately known.

'makelist(i)': Creates a new list containing index i.

'merge(p1, p2)': Combines two jump lists p1 and p2.

'backpatch(p, target_label)': Fills target_label as the destination for all jump instructions indexed in list p.

Topic 6 & 7: Translation of Function Calls & Returns

Standard Calling Sequence Three-Address Instructions: For high-level call `x = f(a, b);``:

```
param a
param b
call f, 2
t1 = return_value
x = t1
```

For function return ``return y;``:

```
return y
```

Topic 8 & 9: Memory Layout & Activation Records

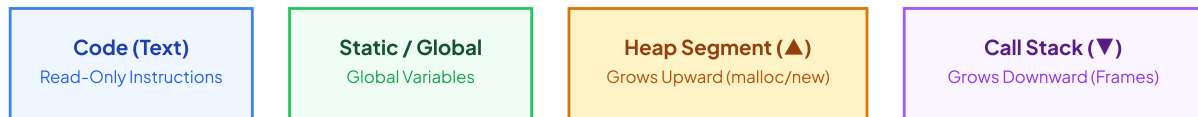


Figure 4.1: Runtime Memory Address Space Layout

🧠 Memory Hook: Activation Record Anatomy (P-R-C-A-S-L-T)

Field	Functional Role in Stack Frame
P — Parameters	Actual argument values passed by the caller.
R — Return Value	Space allocated to return computation result to caller.
C — Control Link	Dynamic link pointing to caller's activation record (restores stack frame on return).
A — Access Link	Static link pointing to enclosing lexical scope for resolving non-local variables.
S — Saved Status	Saved machine state (Program Counter PC, CPU registers).
L — Local Variables	Automatic local variables declared within procedure body.
T — Temporaries	Temporary intermediate expression evaluation registers/values.

M4 Active Recall & Exam-Important Question Bank

Q1. Explain the difference between Control Link (Dynamic Link) and Access Link (Static Link) in Activation Records. (8 Marks)

- **Control Link (Dynamic Link):** Points to the activation record of the *calling procedure* (who called me). It is determined strictly at runtime based on the dynamic execution call stack and is used to restore the caller's stack frame upon return.
- **Access Link (Static Link):** Points to the activation record of the *statically enclosing procedure* in the source code text (where was lexically defined). It is determined at compile time based on lexical block nesting and is used by nested functions to access non-local variables.

☰ Module V: Code Generation & Optimization — Complete 12-Topic Syllabus Tracker

Topic 1: Addresses of Code and Data in Assembly Code

Topic 3: Construction of Basic Blocks (3 Leader Rules)

Topic 5: Machine-Independent Local Optimizations

Topic 7: Unreachable Code Elimination

Topic 9: Constant Propagation Optimization

Topic 11: Common Subexpression Elimination (CSE)

Topic 2: Correlation of Assembly Code with Source Code

Topic 4: Control Flow Graph (CFG Nodes & Edges)

Topic 6: Machine-Independent Global Optimizations

Topic 8: Constant Folding Optimization

Topic 10: Loop-Invariant Code Motion (Hoisting)

Topic 12: Dead Code Elimination

Topic 1 & 2: Assembly Code Addresses & Source Correlation

The final code generator translates intermediate three-address representations into target machine instructions (assembly). The compiler maintains a strict correspondence between source language abstractions, intermediate variables, and physical hardware addresses:

```
Source Code: x = a + b;  
             ↓  
Intermediate: t1 = a + b; x = t1;  
             ↓  
Assembly:    MOV R1, a ; ADD R1, b ; MOV x, R1
```

Symbolic Address Mapping: Source identifiers are assigned relative stack frame offsets or static data segment addresses.

Source-Level Debugging: Line number tables and DWARF metadata allow debuggers (GDB) to map assembly crash addresses directly to source code lines.

Topic 3 & 4: Construction of Basic Blocks & Control Flow Graphs (CFG)

Algorithm: Partitioning Three-Address Code into Basic Blocks (3 Leader Rules)

Step 1: Identify Leaders (First instruction of a basic block):

1. The first three-address instruction of the intermediate code is a Leader.
2. Any instruction that is the target of a conditional or unconditional jump (‘goto L’) is a Leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a Leader.

Step 2: Form Basic Blocks:

For each Leader, its Basic Block consists of the Leader and all subsequent instructions up to (but not including) the next Leader or the end of the program.

Step-by-Step Solved Problem: Basic Block Partitioning Trace

```
(1) a = b + c      ← Leader (Rule 1: First instruction)
(2) if a < d goto 5
(3) x = y + z      ← Leader (Rule 3: Follows conditional jump at line 2)
(4) goto 6
(5) x = 0          ← Leader (Rule 2: Target of goto 5 in line 2)
(6) print x        ← Leader (Rule 2: Target of goto 6 in line 4)
```

Resulting Basic Blocks:

- **Block B_1** : Instructions (1) and (2)
- **Block B_2** : Instructions (3) and (4)
- **Block B_3** : Instruction (5)
- **Block B_4** : Instruction (6)

Topic 5 & 6: Local vs. Global Optimizations

Scope	Operating Domain & Algorithms	Key Advantages & Tradeoffs
Local Optimization	Operates strictly within a single basic block (straight-line code without branches). Analyzed via Directed Acyclic Graphs (DAG).	Fast, linear time complexity; requires zero data-flow analysis equations across blocks.
Global Optimization	Operates across multiple basic blocks throughout the entire Control Flow Graph (CFG). Uses iterative Data-Flow Analysis equations.	Achieves massive performance gains across loops and procedures; higher compile-time cost.

Topic 7 – 12: Principal Machine-Independent Optimizations

Optimization Pass	Transformation Rule	Before → After Example
7. Unreachable Code Elimination	Deletes basic blocks and instructions that have zero incoming edges in the CFG.	<code>`return x; y = 10;`</code> → <code>`return x;`</code>
8. Constant Folding	Evaluates operations on compile-time constants upfront at compile time.	<code>`x = 10 * 20 + 4`</code> → <code>`x = 204`</code>
9. Constant Propagation	Substitutes known constant variable values into subsequent expressions.	<code>`pi = 3.14; r = 2; a = pi * r;`</code> → <code>`a = 3.14 * 2;`</code>
10. Loop-Invariant Code Motion	Hoists computations that evaluate to the exact same value in every loop iteration outside the loop header.	<code>`while (i < n) { x = a * b; i++; }`</code> → <code>`x = a * b; while (i < n) { i++; }`</code>
11. Common Subexpression Elimination (CSE)	Eliminates redundant re-evaluations of expressions whose operand values have not changed.	<code>`t1 = a + b; ...; t2 = a + b;`</code> → <code>`t2 = t1;`</code>
12. Dead Code Elimination	Removes assignments whose computed values are never subsequently read or used anywhere in the program.	<code>`x = 10; x = 20; print(x);`</code> → <code>`x = 20; print(x);`</code>

Q1. Differentiate between Unreachable Code and Dead Code with code examples. (6 Marks)

- **Unreachable Code:** Code that can *never be executed* at runtime because no control flow path can ever reach it (e.g., instructions following an unconditional `return` or `goto` without a label):

`return 5; x = 10;` → `x = 10` is unreachable.

- **Dead Code:** Code that *is executed*, but its computation is useless because the computed value is never referenced by any subsequent instruction:

`x = a + b; x = 20; print(x);` → `x = a + b` executes, but is completely dead because `x` is immediately overwritten before being read.